Assignment A2 : Evaluation and Error Analysis

Individual assignment • 20% individual-report component Released: 26 Nov 2026 Due: 3 Dec 2026, 23:59

Objective. A single accuracy number hides what matters. You will evaluate three supplied models properly — with confidence intervals and per-subgroup breakdowns — and build a structured error taxonomy, so you can argue which model to *deploy* and *for whom it fails*.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.*

Scope

Required core: core metrics (macro/micro P/R/F1), per-subgroup F1 for all three models, and the 20-row error-taxonomy CSV

Optional: extending the error taxonomy — more than 20 errors, or finer linguistic categories.

1. Tasks

You are given three models' predictions (`load_predictions()`, columns `id,pred`) over the **validation** split — for which you have the gold toxicity label and the identity/subgroup columns.

1. **Core metrics.** For each model compute precision, recall, and F1, both **macro** and **micro** averaged.
2. **Fairness.** Compute **per-subgroup F1** for each model and report the gap between the best- and worst-served subgroup.
3. **Error taxonomy.** For one model of your choice, fill a fixed-format CSV of **twenty errors**, each tagged with a `category` (e.g. *negation*, *ambiguous*, *label noise*, *rare term*...). Use **at least three** distinct categories.

Runs on your laptop

Pure `pandas` + `scikit-learn`; runs in seconds. No model training, no GPU. The only real work is doing the statistics and the error reading honestly.

Deliverable & auto-grading

Submit two files:

- `a2_metrics.json` — a top-level `"models"` object mapping each model name to `{precision, recall, f1}` (each of those a `{macro, micro}` pair) plus `f1_by_subgroup`. A top-level summary field (e.g. the least-equitable model) is welcome.
- `a2_errors.csv` — columns `id,gold,pred,category`, exactly **20 rows**.

Grading: metric values checked against the hidden reference within `1e-3`; a script checks `a2_errors.csv` has 20 valid rows with $\geq$3 categories and valid `id`s.

Get this right. In single-label multiclass, **micro-F1 = micro-precision = micro-recall = accuracy** — so your three "micro" numbers are just accuracy; don't read them as extra signal. Also derive the subgroup values from the data (don't hard-code `g0`/`g1`).

2. Findings prompt

Which model would you deploy, and for whom would it fail? Ground your answer in the subgroup F1 gaps — not in the headline accuracy.

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them.* Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A2_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.